2026-8-11 Linux线程--控制

- 1. 线程控制的对象
  - 1.1 从 pthread_t 找到线程
    - 1.1.1 线程控制接口面对的线程标识
    - 1.1.2 用户态 TCB 与内核 LWP 的分工
- 2. 线程终止
  - 2.1 终止单个线程的三种方式
    - 2.1.1 从线程入口函数返回
    - 2.1.2 调用 pthread_exit 终止自身
    - 2.1.3 调用 pthread_cancel 请求取消
  - 2.2 线程结束不等于资源已经回收
    - 2.2.1 执行流结束与 TCB 销毁是两个阶段
    - 2.2.2 退出结果保存在哪里
- 3. 线程等待
  - 3.1 pthread_join 的作用
    - 3.1.1 阻塞等待并回收线程资源
    - 3.1.2 获取线程的执行结果
  - 3.2 结果指针的生命周期
    - 3.2.1 不要返回线程栈上的局部对象
    - 3.2.2 由谁负责释放结果对象
- 4. 线程分离
  - 4.1 joinable 与 detached 是互斥状态
    - 4.1.1 分离让线程结束后的资源自动回收
    - 4.1.2 可以由其他线程分离，也可以由线程自身分离
  - 4.2 分离不会延长进程生命周期
    - 4.2.1 detached 只改变回收责任
    - 4.2.2 选择 join 还是 detach
- 5. 线程控制的分层
  - 5.1 用户态线程库与内核轻量级进程
    - 5.1.1 pthread 库负责线程对象管理
    - 5.1.2 运行时线程库的实现位置
  - 5.2 C++ 线程接口与 pthread 的关系

1. 线程控制的对象

1.1 从 `pthread_t` 找到线程

1.1.1 线程控制接口面对的线程标识

完成线程创建以后，后续控制操作通常都要围绕一个 `pthread_t` 展开，例如：

- 终止线程自身。
- 请求取消另一个线程。
- 等待目标线程结束并获取结果。
- 将目标线程设置为分离状态。

在 Linux 的 NPTL 实现中，`pthread_t` 通常不是内核的 LWP，而是线程库分配在线程虚拟地址空间中的线程描述对象的地址。用户拿着这个标识调用 `pthread` 库接口，线程库再据此找到对应线程的控制信息。

下面这张图说明 `pthread_create` 属于 POSIX 线程库接口，它既负责建立线程，也会把新线程的 `pthread_t` 返回给调用者：

```
pthread_create(3)

NAME
       pthread_create - create a new thread

LIBRARY
       POSIX threads library (libpthread, -lpthread)

SYNOPSIS
       #include <pthread.h>

       int pthread_create(pthread_t *restrict thread,
                          const pthread_attr_t *restrict attr,
                          void *(*start_routine)(void *),
                          void *restrict arg);
```

因此，线程控制的第一步不是直接操作内核中的某个编号，而是先通过 `pthread_t` 找到线程库维护的控制对象。

1.1.2 用户态 TCB 与内核 LWP 的分工

线程库通常会在进程的共享映射区中为每个线程准备一个 `struct pthread`，其中保存线程属性、线程局部存储、线程栈信息以及线程函数的结果等数据。这个结构可以看作用户态线程控制块 TCB。

与此同时，内核会为这个线程建立能够被独立调度的轻量级进程，也就是一个对应的 `task_struct`。在当前 Linux NPTL 模型中，一个用户态 TCB 通常与一个内核 LWP 构成一

对一关系：

> `pthread_t` -> `struct pthread`（用户态 TCB） -> 一个内核 LWP

当循环创建多个线程时，每个线程都会得到不同的 `pthread_t`，但它们仍然属于同一个进程，因此打印出的 PID 相同：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
我是新线程tid:  threadname,tid 0x7e97f27ff6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f1ffe6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f17fd6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f0ffc6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97e85ff6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97eb7fe6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97eaffd6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97ea7fc6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97e9ffb6c0,pid: 973290
```

这说明线程控制需要同时理解两个层次：

1. 用户通过 `pthread_t` 和线程库管理线程对象。
2. 内核通过 LWP 和 `task_struct` 调度真正的执行流。

2. 线程终止

2.1 终止单个线程的三种方式

如果目标只是结束某个线程，而不是结束整个进程，常见方式有三种：

1. 线程入口函数执行到 `return`。
2. 线程调用 `pthread_exit` 终止自身。
3. 一个线程调用 `pthread_cancel` 请求取消同一进程中的另一个线程。

2.1.1 从线程入口函数返回

线程入口函数正常返回后，该线程的执行流结束。线程函数返回的 `void *` 值可以作为线程的退出结果，后续由 `pthread_join` 获取。

需要区分线程函数返回和主函数返回：

- 子线程从自己的入口函数返回，只结束该线程。

- 主线程从 `main` 返回，效果等同于结束整个进程。
- 主线程结束后，进程中的其他线程也会随进程一起结束。

所以“从函数返回”并不能简单理解为“结束任意线程”。它作用于当前线程，但主线程返回还会触发进程级退出。

2.1.2 调用 `pthread_exit` 终止自身

`pthread_exit` 用于终止调用它的线程，而不是终止整个进程。图片给出了对应接口说明：

```
pthread_exit(3)

NAME
       pthread_exit - terminate calling thread

LIBRARY
       POSIX threads library (libpthread, -lpthread)

SYNOPSIS
       #include <pthread.h>

       [[noreturn]] void pthread_exit(void *retval);
```

调用时传入的参数会成为该线程的退出结果，之后可以由其他线程通过 `pthread_join` 获取。例如传入空指针，就表示该线程没有有意义的返回值。

这里要明确两个函数的边界：

- `pthread_exit`：只结束当前线程。
- `exit`：结束整个进程，进程内所有线程随之结束。

线程通过 `pthread_exit` 返回的指针不能指向即将失效的局部栈对象。线程结束后，其栈可能被回收，其他线程再解引用该地址就会产生悬空指针。退出结果应指向生命周期足够长的全局对象、堆对象，或其他由调用双方明确管理的内存。

2.1.3 调用 `pthread_cancel` 请求取消

`pthread_cancel` 作用于另一个线程，通过目标线程的 `pthread_t` 发出取消请求。它与正常返回的区别在于：目标线程不是主动执行到函数末尾，而是响应取消请求后退出。

默认的取消方式是延迟取消。发出请求后，目标线程通常要到取消点才真正结束，例如阻塞等待、休眠或其他允许响应取消的操作。取消是否生效还会受到目标线程取消状态和取消类型的影响，因此不能把它理解为“调用后立即无条件抹掉线程”。

如果线程因取消而结束，等待者通过 `pthread_join` 得到的终止状态通常是 `PTHREAD_CANCELED`。取消属于受控的线程终止方式；除零、野指针等异常则可能破坏整个进程，两者不能混为一谈。

2.2 线程结束不等于资源已经回收

2.2.1 执行流结束与 TCB 销毁是两个阶段

线程函数返回或线程被取消后，线程的执行流已经结束，但线程资源不一定立刻消失。尤其是在默认可等待状态下，线程库还需要保留 TCB 和退出结果，等待其他线程回收。

这就是板书中强调“线程执行流可以退出，但 TCB 可以暂时保留”的原因。一个线程是否还能被等待、退出结果是否还能读取，取决于它结束后的状态，而不只取决于执行流是否已经停止。

2.2.2 退出结果保存在哪里

线程函数的返回值最终会记录在线程库维护的 TCB 中。`pthread_join` 并不是重新执行线程，也不是从已经消失的线程栈上读取局部变量，而是根据 `pthread_t` 找到仍然保留的线程描述对象，再把其中保存的结果交给等待者。

因此，终止和等待构成前后衔接的两个控制阶段：

> 线程结束 -> 执行流停止、TCB 保留结果 -> 等待者调用 join -> 获取结果并回收资源

3. 线程等待

3.1 `pthread_join` 的作用

3.1.1 阻塞等待并回收线程资源

默认创建的线程是可等待的，也就是 joinable 状态。如果不关心其退出结果，也没有把线程设置为分离状态，就应当调用 `pthread_join` 对它进行回收。

`pthread_join` 会阻塞当前线程，直到指定线程结束。它主要完成两件事：

1. 等待目标线程结束。
2. 回收目标线程仍然保留的资源。

如果线程已经结束但一直未被等待，它的 TCB、栈等资源可能继续占用进程资源。这种“线程已经结束，但资源尚未回收”的情况经常被类比为线程层面的僵尸问题，不过它不是进程意义上的僵尸进程。

3.1.2 获取线程的执行结果

`pthread_join` 的第二个参数是一个输出参数，用来接收目标线程的退出结果。目标线程通过 `return` 或 `pthread_exit` 返回的 `void *`，最终会通过这个参数交给等待者。

不同终止方式对应的结果如下：

目标线程终止方式	`pthread_join` 得到的结果
从线程函数 `return`	线程函数的返回值
调用 `pthread_exit(value)`	传入 `pthread_exit` 的 `value`
被 `pthread_cancel` 成功取消	`PTHREAD_CANCELED`
调用者不关心结果	给 `pthread_join` 传入空指针

线程的退出结果不一定是整数，也可以是指向结构体、任务对象或其他数据的指针。板书中通过任务对象说明：线程完成任务后，可以把自己产生的结果作为 `void *` 返回，再由 `pthread_join` 取回。

3.2 结果指针的生命周期

3.2.1 不要返回线程栈上的局部对象

线程函数返回的是一个指针，而不是把指针指向的内容自动复制到等待者手中。以下两类对象不适合作为退出结果：

- 当前线程栈上的局部变量。
- 在线程结束前已经销毁的对象。

线程结束并完成资源回收后，原本的线程栈可能失效。等待者即使拿到了地址，也可能访问已释放或已被复用的内存。

3.2.2 由谁负责释放结果对象

如果退出结果指向堆对象，通常由等待者读取内容后负责释放；如果它指向全局对象或长期存在的对象，则要按照该对象的生命周期管理规则处理。

线程控制不仅包括“把值传回来”，还包括明确结果对象在终止之后仍然有效，以及最终由谁释放。否则即使 `pthread_join` 成功返回，也可能产生悬空指针或内存泄漏。

4. 线程分离

4.1 joinable 与 detached 是互斥状态

4.1.1 分离让线程结束后的资源自动回收

如果一个线程的结果对主线程并不重要，那么每次都调用 `pthread_join` 进行阻塞等待可能没有必要。此时可以把目标线程设置为分离状态。

线程被分离后，它退出时由系统自动处理线程资源，不再要求其他线程对其执行 `pthread_join`。分离状态解决的是“线程结束以后由谁回收资源”，而不是让线程运行得更快。

当一个线程已经处于 joinable 状态时，它不能同时保持为 detached 状态。已经分离的线程也不应再被 `pthread_join` 等待，否则接口会失败或产生不确定行为。

4.1.2 可以由其他线程分离，也可以由线程自身分离

分离操作仍然需要目标线程的 `pthread_t`。常见方式包括：

- 由主线程或其他线程分离目标线程。
- 目标线程使用自己的 `pthread_t` 分离自身。

如果需要等待线程结果，选择 joinable；如果不需要结果，也不希望调用者阻塞等待，选择 detached。两种策略对应不同的资源回收方式，不能只创建线程却既不 join 也不 detach。

4.2 分离不会延长进程生命周期

4.2.1 detached 只改变回收责任

分离线程仍然依赖所在进程的生命周期。若主线程从 `main` 返回，导致整个进程退出，分离线程同样会结束。线程分离只能让线程结束后自动回收自己的线程资源，不能保证进程继续存活到该线程完成。

如果希望主线程退出后其他线程继续运行，应在主线程中使用 `pthread_exit` 结束自身，而不是直接从 `main` 返回。即使如此，最后一个线程结束或进程被其他方式终止时，整个进程仍然会结束。

4.2.2 选择 join 还是 detach

控制方式	是否阻塞等待	能否获取退出结果	资源回收方式
`pthread_join`	会等待目标线程结束	可以	等待者回收
`pthread_detach`	不负责等待结果	不再通过 join 获取	线程结束时自动回收

线程控制策略应当在线程结束前明确。需要结果就 join，不需要结果且不希望阻塞就 detach；既不考虑结果又保持 joinable，容易留下未回收资源。

5. 线程控制的分层

5.1 用户态线程库与内核轻量级进程

5.1.1 `pthread` 库负责线程对象管理

`pthread_t`、`struct pthread`、线程栈、线程局部存储和退出结果等，主要由线程库管理。`pthread_exit`、`pthread_join` 和 `pthread_detach` 等控制接口首先操作这些用户态线程数据，再与内核调度实体配合完成线程生命周期控制。

因此，线程控制不是单纯操作一个 PID：

- `pthread_t` 用于在线程库中找到目标线程。
- TCB 保存控制信息、线程属性和退出结果。
- LWP 是内核实际调度的轻量级进程。

5.1.2 运行时线程库的实现位置

线程控制接口由运行时线程库提供。不同 glibc 版本和发行版中，`libpthread` 可能显示为独立动态库，也可能已经被整合进 `libc`，因此使用 `ldd` 查看程序依赖时不一定能单独看到 `libpthread`；这不影响 `pthread` 系列接口的语义。

课堂手记中的依赖检查结果如下：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ldd test_Thread
        linux-vdso.so.1 (0x00007fffc85e4000)
        libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x0000730441600000)
        libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000730441977000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000730441200000)
        libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x000073044188e000)
        /lib64/ld-linux-x86-64.so.2 (0x00007304419b5000)
ZL@VM-0-14-ubuntu:~/code/lesson30$ ls -l /lib/x86_64-linux-gnu/libc.so.6
-rwxr-xr-x 1 root root 2125328 Jan 30  2026 /lib/x86_64-linux-gnu/libc.so.6
ZL@VM-0-14-ubuntu:~/code/lesson30$
```

理解线程控制时，关键不是记住某个版本中线程库文件叫什么，而是分清用户态线程库管理与内核轻量级进程调度这两个层次。

5.2 C++ 线程接口与 pthread 的关系

线程控制的底层语义不会因为改用 C++ 而改变。C++ 标准线程接口通常会对平台提供的原生线程接口进行封装，在 Linux 上，其底层仍然要完成创建线程、等待结束、分离和回收资源等工作。

封装可以隐藏不同操作系统之间的接口差异，并向用户提供对象化、跨平台的统一接口，但它不能消除以下核心问题：

- 线程结束后是否需要回收。
- 是否需要等待线程结果。
- 退出结果指向的对象是否仍然有效。
- joinable 和 detached 状态是否发生冲突。

因此，先理解 `pthread` 层线程控制的生命周期，再学习 C++ 线程封装，能够更清楚地知道上层对象在底层实际做了什么。

线程控制的整体生命周期可以概括为：

> 创建线程 -> 执行任务 -> return、`pthread_exit` 或取消 -> join 回收并获取结果，或者 detach 后自动回收